

Operating System Lab Assignment-2

Implementation of Banker's Algorithm for Deadlock Avoidance

Student Details

Name: Diya Kumal
Course: BSC Computer Science
Subject: Operating System
Course Code: ETCCOS204

Aim

To implement Banker's Algorithm to check whether the system is in a safe state and determine the safe sequence.

Introduction

Deadlock is a condition where processes cannot proceed because they are waiting for resources held by others. Banker's Algorithm avoids deadlock by ensuring the system stays in a safe state.

Objectives

- Understand resource allocation
- Calculate Need Matrix
- Implement Banker's Algorithm
- Determine safe sequence
- Analyze system safety

Algorithm

1. Input data
2. Calculate Need = Max - Allocation
3. Apply safety algorithm
4. Find safe sequence

Python Code

Banker's Algorithm

```
def bankers_algorithm(processes, resources, allocation, maximum, available):
```

```
    # Calculate Need Matrix
```

```
    need = []
```

```
    for i in range(processes):
```

```
        row = []
```

```
        for j in range(resources):
```

```
            row.append(maximum[i][j] - allocation[i][j])
```

```
        need.append(row)
```

```
    print("\nNeed Matrix:")
```

```
    for row in need:
```

```
        print(row)
```

```
    finish = [False] * processes
```

```
    safe_sequence = []
```

```
    work = available.copy()
```

```
    while len(safe_sequence) < processes:
```

```
        found = False
```

```
        for i in range(processes):
```

```
            if not finish[i]:
```

```
                if all(need[i][j] <= work[j] for j in range(resources)):
```

```
                    # Allocate resources
```

```
                    for j in range(resources):
```

```
work[j] += allocation[i][j]
```

```
safe_sequence.append(i)
```

```
finish[i] = True
```

```
found = True
```

```
if not found:
```

```
    print("\nSystem is NOT in safe state")
```

```
    return
```

```
print("\nSystem is in SAFE state")
```

```
print("Safe Sequence:", safe_sequence)
```

```
# Example Input
```

```
processes = 5
```

```
resources = 3
```

```
allocation = [
```

```
    [0, 1, 0],
```

```
    [2, 0, 0],
```

```
    [3, 0, 2],
```

```
    [2, 1, 1],
```

```
    [0, 0, 2]
```

```
]
```

```
maximum = [  
    [7, 5, 3],  
    [3, 2, 2],  
    [9, 0, 2],  
    [2, 2, 2],  
    [4, 3, 3]  
]
```

```
available = [3, 3, 2]
```

```
bankers_algorithm(processes, resources, allocation, maximum, available)
```

Output

Need Matrix:

```
[7, 4, 3]
```

```
[1, 2, 2]
```

```
[6, 0, 0]
```

```
[0, 1, 1]
```

```
[4, 3, 1]
```

System is in SAFE state

Safe Sequence: [1, 3, 4, 0, 2]

Conclusion

Banker's Algorithm helps avoid deadlock by ensuring safe execution sequence.